

Chapter 7: The multivariate normal model

Jesse Mu

2016-11-10

Contents

1	The multivariate normal density	1
1.1	Example: reading comprehension	1
1.2	Multivariate normal density	2
2	A semiconjugate prior distribution for the mean	3
3	The inverse-Wishart distribution	4
3.1	Specifying parameters	5
3.2	Full conditional distribution of $\Sigma \mid y_1, \dots, y_n, \theta$	5
4	Summary of inference with the multivariate normal	6
4.1	(Semiconjugate) prior	6
4.2	Posterior	6
5	Gibbs sampling of the mean and covariance	7
5.1	Example: reading comprehension	7
5.1.1	Specifying prior	7
5.1.2	Code	7
6	Missing data and imputation	10
6.1	Gibbs sampling with missing data	10
6.2	Example	11
7	Exercises	14
7.1	7.1	14
7.1.1	a	14
7.1.2	b	14
7.2	7.2	15
7.2.1	a	15
7.2.2	b	16
7.2.3	c	16
7.3	7.3	17
7.3.1	a	17
7.3.2	b	18
7.3.3	c	19
7.3.4	d	20
7.3.5	e	21

1 The multivariate normal density

1.1 Example: reading comprehension

We make the step to two variables by example. Consider a sample ($n = 22$) of children who are given reading comprehension tests before and after receiving a particular instructional method. So

each student has a before and after test score. We can denote these *two* variables for student i as a vector $\mathbf{Y}_i$, where $Y_{i,1}$ is the before score, and $Y_{i,2}$ is the after score:

$$\mathbf{Y}_i = \begin{pmatrix} Y_{i,1} \\ Y_{i,2} \end{pmatrix} \quad (1)$$

Recall

$$\text{Cor}(X, Y) = \frac{\text{Cov}(X, Y)}{\text{SD}(X)\text{SD}(Y)} \quad (2)$$

$$= \frac{\sigma_{1,2}}{\sigma_1\sigma_2} \quad (3)$$

$$= \frac{\sigma_{1,2}}{\sqrt{\sigma_1^2\sigma_2^2}} \quad (4)$$

1.2 Multivariate normal density

If

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{pmatrix} \sim \mathcal{N}(\theta, \Sigma) \quad (5)$$

where

$$\theta = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{pmatrix} \quad (6)$$

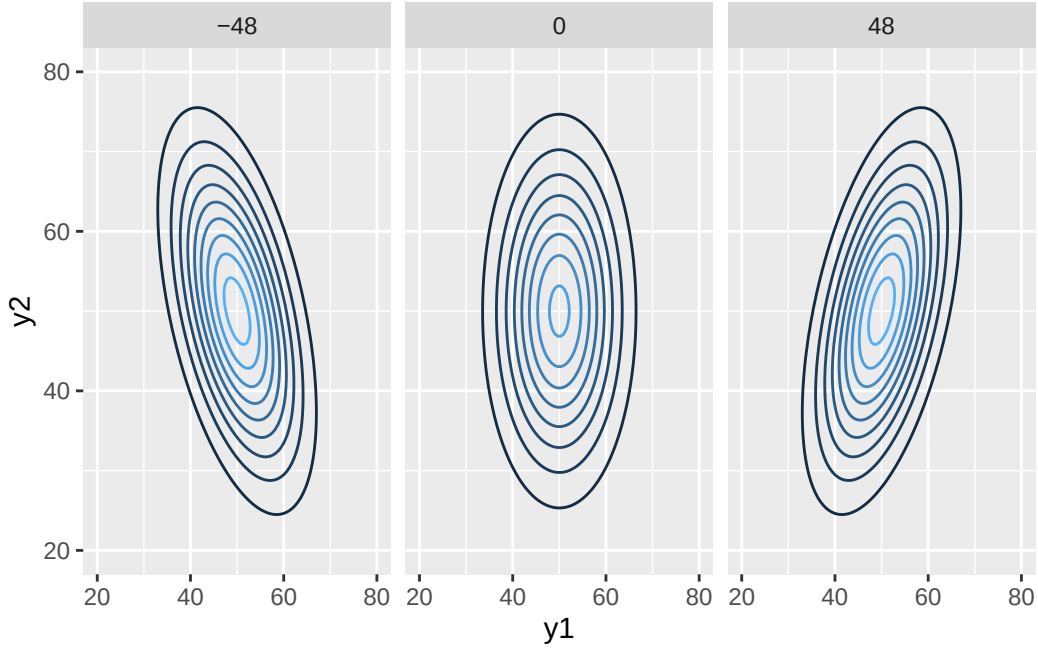
and

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{1,2} & \cdots & \sigma_{1,p} \\ \sigma_{1,2} & \sigma_2^2 & \cdots & \sigma_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1,p} & \cdots & \cdots & \sigma_p^2 \end{pmatrix} \quad (7)$$

then

$$p(\mathbf{y} \mid \theta, \Sigma) = (2\pi)^{-p/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2} (\mathbf{y} - \theta)^T \Sigma^{-1} (\mathbf{y} - \theta)\right) \quad (8)$$

It's useful to compare this to the single variable case. The first two terms represent $1/\sqrt{2\pi\sigma^2}$ in the univariate case, generalized to an arbitrary dimension: there is no longer the square root of the variance for a single variable, but rather a p -dimensional covariance matrix raised to the $1/2$ power. Furthermore, the quadratic term in the exponential $(y - \theta)^2$ can be seen in the exponential here, but also included is the matrix multiplication with the inverse of the covariance matrix.



2 A semiconjugate prior distribution for the mean

At this point we have given up on computing a full closed form solution for the posterior, as it's too complicated. Rather, if we compute conjugate priors and posteriors for the full conditional distributions of the θ and Σ separately, we can use Gibbs sampling to easily estimate the joint posterior distribution. Let's start by calculating θ first:

Analogous the univariate case, for the multivariate normal distribution, a conjugate prior for the population mean is a multivariate normal.

Let μ_0 be the prior mean, and Λ_0 be the covariance matrix of μ_0 . Then let $\theta \sim \mathcal{N}(\mu_0, \Lambda_0)$. What does this prior look like?

$$p(\theta \mid \Sigma) \propto \exp \left[-\frac{1}{2}(\theta - \mu_0)^T \Lambda_0^{-1}(\theta - \mu_0) \right] \quad \text{Drop constants} \quad (9)$$

$$= \exp \left[-\frac{1}{2}(\theta^T - \mu_0^T)(\Lambda_0^{-1}\theta - \Lambda_0^{-1}\mu_0) \right] \quad (10)$$

$$= \exp \left[-\frac{1}{2}(\theta^T \Lambda_0^{-1}\theta - \mu_0^T \Lambda_0^{-1}\theta - \theta^T \Lambda_0^{-1}\mu_0 + \mu_0^T \Lambda_0^{-1}\mu_0) \right] \quad (11)$$

$$= \exp \left[-\frac{1}{2}(\theta^T \Lambda_0^{-1}\theta - 2\theta^T \Lambda_0^{-1}\mu_0 + \mu_0^T \Lambda_0^{-1}\mu_0) \right] \quad \text{Middle terms combine} \quad (12)$$

$$\propto \exp \left[-\frac{1}{2}\theta^T \Lambda_0^{-1}\theta + \theta^T \Lambda_0^{-1}\mu_0 \right] \quad \text{Discard last term} \quad (13)$$

If we let $\mathbf{A}_0 = \Lambda_0^{-1}$, i.e. the precision matrix (which echoes the univariate case) and $\mathbf{b}_0 = \Lambda_0^{-1}\mu_0$, this simplifies to

$$p(\theta) = \exp \left(-\frac{1}{2}\theta^T \mathbf{A}_0 \theta + \theta^T \mathbf{b}_0 \right) \quad (14)$$

$$(15)$$

We will see this simplified form show up when working with the sampling models and posterior distribution as well.

Let's first look at the sampling model. Doing similar simplifications, we get

$$p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \theta, \Sigma) \propto \exp\left(-\frac{1}{2}\theta^T \mathbf{A}_1 \theta + \theta^T \mathbf{b}_1\right) \quad (16)$$

where this time $\mathbf{A}_1 = n\Sigma^{-1}$ and $\mathbf{b}_1 = n\Sigma^{-1}\bar{\mathbf{y}}$. $\bar{\mathbf{y}}$ is the vector of sample averages for each variable.

So finally, we can calculate the posterior for θ :

$$p(\theta \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \Sigma) = p(\theta \mid \Sigma)p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \theta, \Sigma)/p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \Sigma) \quad (17)$$

$$\propto p(\theta \mid \Sigma)p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \theta, \Sigma) \quad (18)$$

$$\propto \exp\left(-\frac{1}{2}\theta^T \mathbf{A}_0 \theta + \theta^T \mathbf{b}_0\right) \times \exp\left(-\frac{1}{2}\theta^T \mathbf{A}_1 \theta + \theta^T \mathbf{b}_1\right) \quad (19)$$

$$= \exp\left(-\frac{1}{2}\theta^T \mathbf{A}_n \theta + \theta^T \mathbf{b}_n\right) \quad (20)$$

where we have combined terms such that $\mathbf{A}_n = \mathbf{A}_0 + \mathbf{A}_1 = \Lambda_0^{-1} + n\Sigma^{-1}$ and $\mathbf{b}_n = \mathbf{b}_0 + \mathbf{b}_1 = \Lambda_0^{-1}\mu_0 + n\Sigma^{-1}\bar{\mathbf{y}}$.

So if $\theta \mid \Sigma \sim \mathcal{N}(\mu_0, \Lambda_0)$, then $\theta \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \Sigma \sim \mathcal{N}(\mu_n, \Lambda_n)$ where the parameters are defined as above.

Then notice that, like the univariate case:

- $\text{Cov}(\theta \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \Sigma) = \Lambda_n = (\Lambda_0^{-1} + n\Sigma^{-1})^{-1}$, a combination of prior and posterior precision, and
- $\mathbb{E}(\theta \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \Sigma) = \mu_n = (\Lambda_0^{-1} + n\Sigma^{-1})^{-1}(\Lambda_0^{-1}\mu_0 + n\Sigma^{-1}\bar{\mathbf{y}})$, weighted average of the prior estimate of the mean and the sample mean.

3 The inverse-Wishart distribution

We've seen the (semi)conjugate prior and posterior distribution for the mean. Now to the covariance matrix Σ . Remember that for the univariate normal distribution, a semi-conjugate prior distribution for the variance was the inverse-Gamma distribution. Similarly, for the multivariate case, a semi-conjugate prior distribution for the covariance matrix is the inverse of the multivariate analog of the Gamma distribution, known as a Wishart distribution.

Generating samples from a Wishart distribution is analogous to sampling a set of variables from a multivariate normal distribution and calculating the empirical covariance matrix of the samples. More specifically, with parameters $\nu_0 \in \mathbb{Z}^+$ and Φ_0 (a $p \times p$ covariance matrix),

1. Sample $z_1, \dots, z_{\nu_0} \sim \mathcal{N}(0, \Phi_0)$
2. Calculate $Z^T Z = \sum_{i=1}^{\nu_0} z_i z_i^T$

Then $Z_1^T Z_1, \dots, Z_S^T Z_S \sim \text{Wishart}(\nu_0, \Phi_0)$.

Some properties of samples from the Wishart which happen to stem from calculating empirical covariance matrices in the first place

- If $\nu_0 > p$ then $Z^T Z$ is positive definite
- $Z^T Z$ is symmetric
- $\mathbb{E}(Z^T Z) = \nu_0 \Phi_0$

Like how the Gamma is not the conjugate prior of the variance for the Normal, the Wishart is not the conjugate prior of the variance for the multivariate Normal; rather, the inverse-Wishart is. Sampling from an inverse-Wishart just involves taking $\Sigma^{(s)} = (Z^{(s)T} Z^{(s)})^{-1}$

If we're sampling from an inverse-Wishart we rename Φ_0 to $\mathbf{S}_0^{-1}$ such that

- $\mathbb{E}(\Sigma^{-1}) = \nu_0 \mathbf{S}_0^{-1}$ (instead of Φ_0)

- $\mathbb{E}(\Sigma) = \frac{1}{\nu_0 - p - 1} \mathbf{S}_0$ (so not exactly the inverse of $\mathbf{S}_0^{-1}$)

3.1 Specifying parameters

Specifying parameters is somewhat interesting for the inverse-Wishart because there are many of them - we need to specify the entire covariance matrix. If we have a prior expectation of a covariance matrix Σ_0 , then we can center our prior around it in two suggested ways:

- Set ν_0 large and set $\mathbf{S}_0 = (\nu_0 - p - 1)\Sigma_0$, such that $\mathbb{E}(\Sigma) = \frac{\nu_0 - p - 1}{\nu_0 - p - 1} \Sigma_0 = \Sigma_0$ and (due to large ν_0) the prior is fairly concentrated around Σ_0 ;
- Set $\nu_0 = p + 2$ and let $\mathbf{S}_0 = \Sigma_0$, such that $\mathbb{E}(\Sigma) = \frac{1}{p+2-p-1} \Sigma_0 = \Sigma_0$ but only loosely centered around Σ_0 (due to fairly small ν_0)

For an “empirical Bayes” approach, we can center our prior around the empirical covariance matrix of our sample.

3.2 Full conditional distribution of $\Sigma \mid y_1, \dots, y_n, \theta$

There is an intimidating normalizing constant for the inverse-Wishart. All we need to know is if $\Sigma \sim \text{inverse-Wishart}(\nu_0, \mathbf{S}_0^{-1})$,

$$p(\Sigma) \propto |\Sigma|^{-(\nu_0 + p + 1)/2} \times \exp(-\text{tr}(\mathbf{S}_0 \Sigma^{-1})/2) \quad (21)$$

Recall the sampling model from earlier. We skipped some simplifications above (substituting $\mathbf{A}_1$ and b_1) so take it for granted that a less-simplified form is

$$p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \theta, \Sigma) = (2\pi)^{-np/2} |\Sigma|^{-n/2} \exp\left(-\sum_{i=1}^n (y_i - \theta)^T \Sigma^{-1} (y_i - \theta)/2\right) \quad (22)$$

Using some linear algebra,

$$\sum_{i=1}^n (y_i - \theta)^T \Sigma^{-1} (y_i - \theta) = \text{tr}\left(\left(\sum_{i=1}^n (y_i - \theta)(y_i - \theta)^T\right) \Sigma^{-1}\right) \text{tr}(\mathbf{S}_\theta \Sigma^{-1}) \quad (23)$$

where $\mathbf{S}_\theta = \sum_{i=1}^n (y_i - \theta)(y_i - \theta)^T$ is the *residual sum of squares matrix* for the vectors $y_1, \dots, y_n$. (To obtain the residual sum of squares matrix, you calculate the sum of squares for the residual vectors $y_i - \theta$)

So

$$p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \theta, \Sigma) = (2\pi)^{-np/2} |\Sigma|^{-n/2} \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1})/2) \quad (24)$$

Now we can calculate the full conditional distribution of Σ :

$$p(\Sigma \mid y_1, \dots, y_n, \theta) \propto p(\Sigma) \times p(y_1, \dots, y_n \mid \theta, \Sigma) \quad (25)$$

$$\propto [|\Sigma|^{-(\nu_0 + p + 1)/2} \times \exp(-\text{tr}(\mathbf{S}_0 \Sigma^{-1})/2)] \times [|\Sigma|^{-n/2} \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1})/2)] \quad (26)$$

$$= |\Sigma|^{-(\nu_0 + n + p + 1)/2} \exp(-\text{tr}((\mathbf{S}_0 + \mathbf{S}_\theta) \Sigma^{-1})/2) \quad (27)$$

$$\propto \text{dinverse-Wishart}(\nu_0 + n, [\mathbf{S}_0 + \mathbf{S}_\theta]^{-1}) \quad (28)$$

$$= \text{dinverse-Wishart}(\nu_n, \mathbf{S}_n^{-1}) \quad (29)$$

where $\nu_n = \nu_0 + n$ and $\mathbf{S}_n = \mathbf{S}_0 + \mathbf{S}_\theta$.

Like the univariate case, the conditional distribution on Σ is dependent on $\nu_0 + n$, a sum of the prior sample size and the data sample size, $\mathbf{S}_0 + \mathbf{S}_\theta$, the sum of the “prior” residual sum of squares and the empirical sum of squares.

Recall that, since inverse-Wishart matrices involve sampling from a normal distribution with mean $\mathbf{0}$, indeed $\mathbf{S}_0$ can be treated as a *residual* covariance matrix, given that $\mathbb{E}(y_i - \theta) = \mathbf{0}$.

Finally, notice that the conditional expectation of the covariance matrix is a weighted average of the prior expectation $\frac{1}{\nu_0 - p - 1} \mathbf{S}_0$ and the unbiased estimator $\frac{1}{n} \mathbf{S}_\theta$:

$$\mathbb{E}(\Sigma \mid y_1, \dots, y_n, \theta) = \frac{1}{\nu_0 + n - p - 1} (\mathbf{S}_0 + \mathbf{S}_\theta) \quad (30)$$

$$= \frac{\nu_0 - p - 1}{\nu_0 + n - p - 1} \frac{1}{\nu_0 - p - 1} \mathbf{S}_0 + \frac{n}{\nu_0 + n - p - 1} \frac{1}{n} \mathbf{S}_\theta. \quad (31)$$

So the Bayesian estimator is a biased estimator, but is still demonstrably consistent as $n \rightarrow \infty$. As mentioned in Chapter 5, we hope that the estimator is biased more towards the true mean, as long as the prior is mildly informative.

4 Summary of inference with the multivariate normal

Like in Chapter 5, here we summarize the moving parts of inference with the multivariate normal sampling model. There are four prior parameters (note some are matrices):

4.1 (Semiconjugate) prior

- $\mathbf{S}_0$ for the inverse-Wishart
 - *related to* the prior estimate of the covariance between the variables
 - Only *related to* because as mentioned above, there are some guidelines for what to use for ν_0 and $\mathbf{S}_0$ such that the prior distribution is centered around Σ_0 , the *true* prior estimate of the covariance matrix you are looking for.
- ν_0 for the inverse-Wishart
 - a “prior sample size” from which the initial estimate of the *variance* is observed
- μ_0 for the multivariate normal
 - an initial estimate for the population mean
- Λ_0 for the multivariate normal
 - the covariance (i.e. uncertainty) of the initial estimate for the population mean

Somewhat similar to the univariate case, the estimate of the covariance matrix for the inverse-Wishart prior is decoupled from the estimate of the covariance of the mean vector in the multivariate normal prior, although it’s common to set these the same (but again, see rules for determining ν_0).

Note that this is somewhat *different* than the univariate case; since there were no covariances to worry about, what was decoupled was “prior sample sizes” from which the prior variance and prior mean are observed. Like here, it was also common to set these the same.

4.2 Posterior

The updated parameters are

- $\mathbf{S}_n = \mathbf{S}_0 + \mathbf{S}_\theta$, where $\mathbf{S}_\theta$ is the residual sum of squares matrix
- $\nu_n = \nu_0 + n$
- $\mu_n = (\Lambda_0^{-1} + n\Sigma^{-1})^{-1} (\Lambda_0\mu_0 + n\Sigma^{-1}\bar{y}) = \Lambda_n(\Lambda_0^{-1}\mu_0 + n\Sigma^{-1}\bar{y})$
- $\Lambda_n = (\Lambda_0^{-1} + n\Sigma^{-1})^{-1}$

5 Gibbs sampling of the mean and covariance

Knowing these values, we can now perform Gibbs sampling to sample from $p(\theta, \Sigma \mid y_1, \dots, y_n)$. Specifically, we start with an estimate of one of the two values - $\Sigma^{(0)}$ for simplicity - and use the following algorithm:

1. Sample $\theta^{(s+1)} \sim \mathcal{N}(\mu_n, \Lambda_n)$ where the parameters are calculated as above. This depends on the inverse of the previous $\Sigma^{(s)}$.
2. Sample $\Sigma^{(s+1)} \sim \text{inverse-Wishart}(\nu_n, \mathbf{S}_n^{-1})$, where the parameters depend on $\theta^{(s+1)}$.

5.1 Example: reading comprehension

We return to the example of two reading comprehension exams, given pre- and post-training.

5.1.1 Specifying prior

Mean

Assume the tests are designed such that people generally score 50 out of 100. Thus, our prior mean is $\mu_0 = (50, 50)^T$. Note that this implicitly assumes there is no effect of the training - the prior expectation of the post-training score is the same as the pre-training score.

Next, we need to specify the covariance of the prior expectation of the mean. Specifically, since the scores are bounded between 0 and 100, we should put little probability outside the $[0, 100]$ range (so a bell curve centered on 50 with 2 standard deviations $\in [0, 100]$). Then 1 standard deviation is 25 and the variance is thus 625.

Since the scores are both testing reading ability the scores are probably correlated. So if we want a prior correlation between θ_1 and θ_2 of 0.5, we need to solve the correlation equation

$$0.5 = \frac{\sigma_{1,2}}{\sqrt{\sigma_1^2 \sigma_2^2}} \quad (32)$$

$$\Rightarrow \sigma_{1,2} = 0.5 \sqrt{625^2} = 312.5 \quad (33)$$

$$(34)$$

Therefore,

$$\Lambda_0 = \begin{bmatrix} 625 & 312.5 \\ 312.5 & 625 \end{bmatrix}$$

Variance

We will use the guideline mentioned earlier for loosely centering our covariance matrix prior on Λ_0 . We'll set $\mathbf{S}_0^{-1} = \Lambda_0$ and $\nu_0 = p + 2 = 4$.

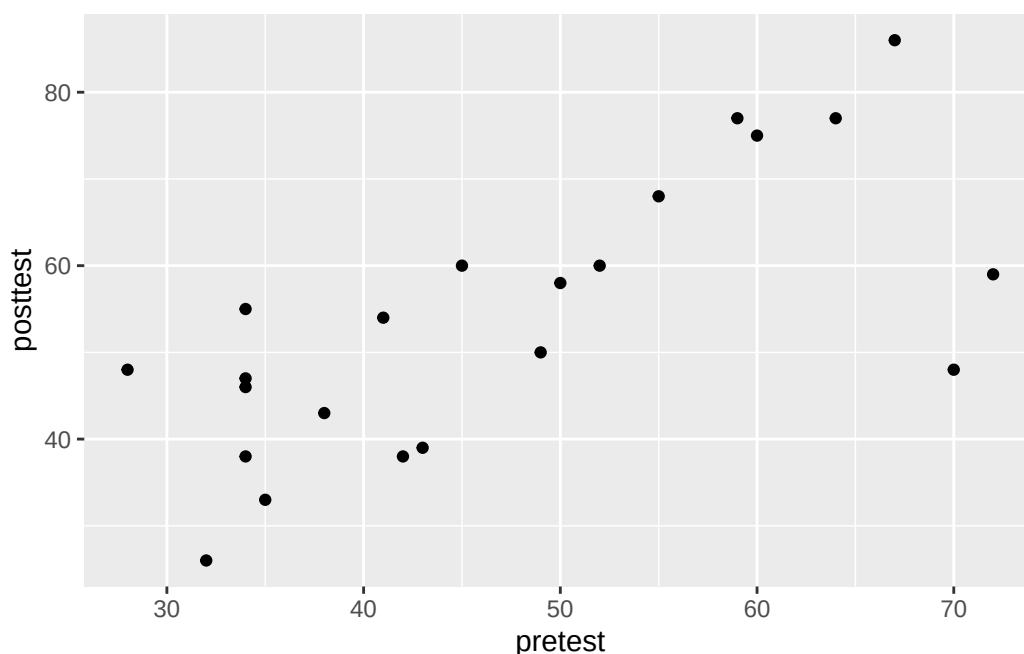
5.1.2 Code

```
# Prior specification
Mu_0 = c(50, 50) # If coerced, will be treated as column
Lambda_0 = rbind(c(625, 312.5), c(312.5, 625))

nu_0 = 4
S_0 = matrix(rep(Lambda_0), nrow = 2)

# Note that this defines rmvnorm and rwish, but I am going to use default
# implementations or packages to see what I would be using in the real world.
# Specifically we need MASS::mvrnorm (Modern Applied Statistics with S), and
# wishart comes default in newer R distributions with rWishart
```

```
source("Inline/chapter7.R")
# Try plotting
Y = Y.reading
ggplot(data.frame(Y.reading)) +
  geom_point(aes(x = pretest, y = posttest))
```



```
# Gibbs sampling
library(MASS)
n = nrow(Y) # Number of observations
Sigma_0 = cov(Y) # Calculate covariance matrix; initial Sigma sample
# NOTE: This initial Sigma sample doesn't end up in the gibbs sample - we throw
# it away and start from scratch where sample 1 is the Theta based on Sigma_0, and
# the Sigma based on that new Theta
Sigma = Sigma_0
ybar = colMeans(Y) # Faster way of calculating column means

# TODO: Preallocate space for these instead of setting as NULL
THETA = NULL
SIGMA = NULL

# Also, inv = solve to make it more readable
inv = solve

set.seed(1)
for (s in 1:5000) {
  # Update theta
  # 1a. Compute params: Mu_n and Lambda_n from y_1, \dots, y_n and \Sigma^{(s)}
  # > Compute Lambda_n according to equation 7.4
  # Note: could cache inverse of priors
  Lambda_n = inv(inv(Lambda_0) + n * inv(Sigma))
  # > Compute Mu_n according to 7.5. Use Matrix mult
  # Note that the first term in 7.5 is Lambda_n
  Mu_n = Lambda_n %*% (inv(Lambda_0) %*% Mu_0 + n * inv(Sigma) %*% ybar)

  # 1b. Sample \theta^{(s+1)} \sim multivariate normal mu_n, lambda_n
  # Now we know Lambda_n, Mu_n as implied by the known Sigma and data (p. 108).
```



```

# Sample theta from multivariate normal (7.6, implied by 7.3)
Theta = mvrnorm(n = 1, Mu_n, Lambda_n)

# Known Theta. Now sample a new Sigma. NOTE: Old sigma gets thrown away!
# 2a) Compute params for Sigma: Compute S_n from data and \theta^{(s+1)}
# i.e. Given the data and this Theta, we need to calculate the parameters that
# define the full conditional distribution of \Sigma^{(s+1)}
# S_n according to p.112 first paragraph - not defined, but could be in 7.9
# S_\theta is the residual sum of squares, defined in unlabeled equation after
# 7.8
# Use vectorized to avoid summation
# Calculate residuals, then do sum of squares
# Q: why t(Y)? just how elementwise works I guess
resid = t(Y) - c(Theta)
S_theta = resid %*% t(resid)
S_n = S_0 + S_theta
# 2b) Knowing the parameters for the full conditional distribution on Sigma
# (inverse wishart with nu_0 and S_n known), sample
# df = number of samples (degrees of freedom)
# Don't forget to invert it afterwards (inverse wishart)
# Weird thing is rWishart returns a weird list of arrays
Sigma = inv(rWishart(1, nu_0 + n, inv(S_n))[, , 1])

THETA = rbind(THETA, Theta)
# Flatten sigma??
SIGMA = rbind(SIGMA, c(Sigma))
}

```

Here are some associated calculations with this sample

```

# Confidence interval for difference between post and pre test
quantile(THETA[, 2] - THETA[, 1], prob = c(0.025, 0.5, 0.975))
#>      2.5%      50%      97.5%
#>  1.454197  6.614914 11.663732

# Likelihood that the mean for the second test is greater than the mean for the
# 1st test
mean(THETA[, 2] > THETA[, 1])
#> [1] 0.9924

# Confidence interval for the correlation
CORR = apply(SIGMA, MARGIN = 1, FUN = function(row) {
  # indices 1 and 4 are correlation of dims 1 and 2
  # indices 2 is equal to index 3 is equal to covariance
  row[2] / sqrt(row[1] * row[4])
})
# Obviously there is a correlation
quantile(CORR, prob = c(0.025, 0.5, 0.975))
#>      2.5%      50%      97.5%
#> 0.4180072 0.6871101 0.8476477

```

We can also work with the posterior predictive distribution by sampling new pairs $(y_1, y_2)^{T(s)}$ from our samples of $\theta^{(s)}$ and $\Sigma^{(s)}$:

```

Y = matrix(0, nrow = 5000, ncol = 2)
for (s in 1:5000) {
  Y[s, ] = mvrnorm(1, THETA[s, ], matrix(SIGMA[s, ], nrow = 2))
}

```

```
# Probability that the post-test score of a randomly selected person is greater
# than the pre-test score
mean(Y[, 2] > Y[, 1])
#> [1] 0.7032
```

Importantly, the probability of an individual having a higher post-test score than pre-test is much lower than the near-1 probability of the difference in mean test scores. It's important to have clarified the difference between population means and individuals when drawing conclusions about your data.

6 Missing data and imputation

Another useful application of Bayesian analysis and the multivariate normal model is the imputation of missing data. There are various naive ways to deal with missing data in datasets:

- listwise deletion: just discard points with missing data. But this wastes valuable data!
- replace missing variables for points with the mean of that variable for the entire dataset. But this results in inaccurate estimates, since a data point's other variables may contain information about the data point's

The Bayesian approach offers a neat solution around this. The idea is that the likelihood of a datapoint with missing values is the likelihood of the observed values while integrating over the missing values.

Specifically, let's assume in a dataset Y with missing values, we have a corresponding matrix O which contains a 1 if the corresponding element in Y exists, else 0. (This is just to help us mathematically indicate which variables are missing)

Then consider a single observation y_i . We have

$$p(o_i, \{y_{i,j} : o_{i,j} = 1\} \mid \theta, \Sigma) = p(o_i) \times \int \left[p(y_i \mid \theta, \Sigma) \prod_{y_{i,j} : o_{i,j}=0} dy_{i,j} \right] \quad (35)$$

i.e. we are integrating over all possible “full” observations of datapoints y with respect to the variables that we don't have. The variables we have observed stay constant.

6.1 Gibbs sampling with missing data

Normally we use Gibbs sampling to estimate the posterior $p(\theta, \Sigma \mid \mathbf{Y})$. Here, however, we don't have a full dataset $\mathbf{Y}$; rather, we have an observed dataset $\mathbf{Y}_{\text{obs}}$ and missing values $\mathbf{Y}_{\text{miss}}$. The key idea is to *also* estimate the posterior distribution on $\mathbf{Y}_{\text{miss}}$, which will also help us make more accurate estimates on θ and Σ . Using Gibbs sampling, when we have sample values $\theta^{(s)}$ and $\Sigma^{(s)}$, we can sample from

$$\mathbf{Y}_{\text{miss}}^{(s)} \sim p(\mathbf{Y}_{\text{miss}} \mid \mathbf{Y}_{\text{obs}}, \theta^{(s)}, \Sigma^{(s)})$$

Specifically, to sample from the above, we simply sample the missing values of each data point independently. For a data point y with missing values, let a be the indices of the observed values and b be the indices of the missing values. Then it is shown that sampling $y_{[b]}$ given known observed variables and the parameters θ and Σ also follows a multivariate normal distribution, but with mean and covariance matrices with dimension $|b|$ that take into account the existing variables:

$$y_{[b]} \mid y_{[a]}, \theta, \Sigma \sim \mathcal{N}(\theta_{b|a}, \Sigma_{b|a}) \quad (36)$$

where

- $\theta_{b|a} = \theta_b + \Sigma_{[b,a]}(\Sigma_{[a,a]})^{-1}(y_{[a]} - \theta_{[a]});$
 - Intuitively, the mean of the multivariate normal distribution on the missing values *given* some observed values starts with the unconditional mean of the observed values, plus or minus some offset that depends on the observed values and the correlations between the observed and missing values. For example, if it is known that a datapoint's observed values are quite high relative to the mean ($y_a - \theta_a$), and that there is a positive correlation between observed values and missing values, we would expect the missing values to generally be higher as well.
- $\Sigma_{b|a} = \Sigma_{[b,b]} - \Sigma_{[b,a]}(\Sigma_{[a,a]})^{-1}\Sigma_{[a,b]}$
 - Intuitively, the covariance matrix of the conditional distribution on the missing values starts with the unconditional covariance, but notice the minus sign; since the covariance matrix is positive definite, knowing about some observed variables will decrease our uncertainty about the missing values.

Once we've sampled a set of missing values, notice that we now have a full "dataset" if we combine our observed values with the newly sampled missing values. This means that we can sample from the full conditional distributions of θ and Σ normally, and from there, once again sample a new set of $\mathbf{Y}_{\text{miss}}$.

To summarize Gibbs sampling with missing data: assume starting values $\Sigma^{(0)}$ and $\mathbf{Y}_{\text{miss}}^{(0)}$ - perhaps the empirical covariance matrix and the unconditional means of the observed sample. Then the algorithm has just one more step:

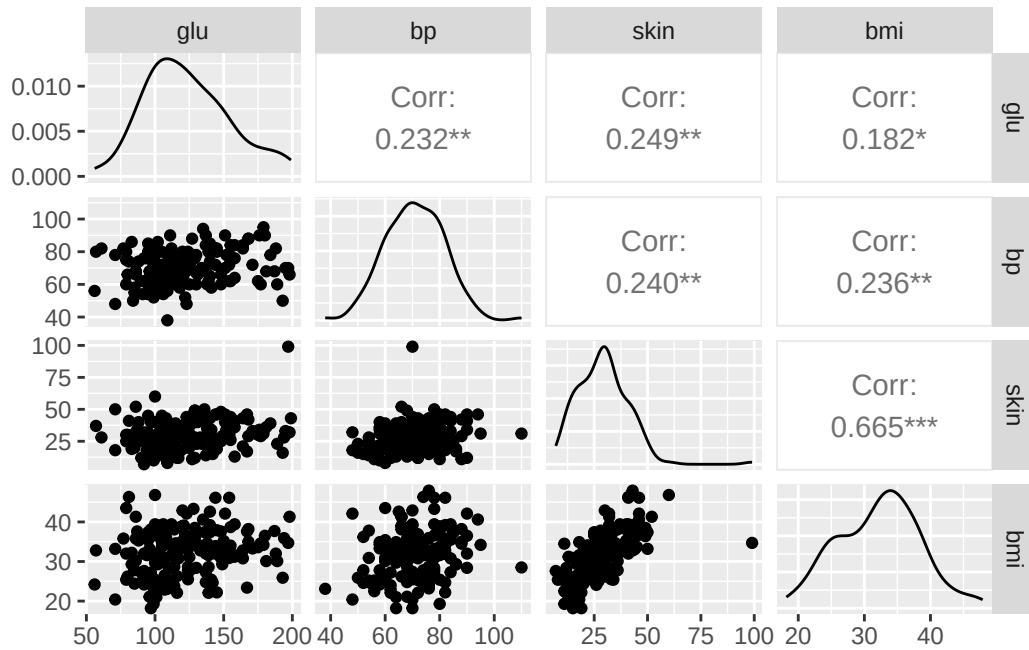
1. Sample $\theta^{(s+1)}$ from $p(\theta | \mathbf{Y}_{\text{obs}}, \mathbf{Y}_{\text{miss}}^{(s)}, \Sigma^{(s)})$
2. Sample $\Sigma^{(s+1)}$ from $p(\Sigma | \mathbf{Y}_{\text{obs}}, \mathbf{Y}_{\text{miss}}^{(s)}, \theta^{(s)})$
3. Sample $\mathbf{Y}_{\text{miss}}^{(s+1)}$ from $p(\mathbf{Y}_{\text{miss}} | \mathbf{Y}_{\text{obs}}, \theta^{(s)}, \Sigma^{(s)})$

For steps 1 and 2, you simply combine the sampled missing data and the observed data for a full dataset $\mathbf{Y}$ and sample from the full conditional distributions like normal.

6.2 Example

Let's take a look at an example using a dataset with four health-related measurements on 200 women near Phoenix, Arizona. Notice that this dataset has missing values. We'll assume that the data is *missing at random*, which is necessary for this analysis.

```
Y = Y.pima.miss
head(Y) # Notice missing data
#> # A tibble: 6 x 4
#>   glu    bp  skin  bmi
#>   <int> <int> <int> <dbl>
#> 1    86    68    28  30.2
#> 2   195    70    33   NA
#> 3    77    82    NA  35.8
#> 4    NA    76    43  47.9
#> 5   107    60    NA   NA
#> 6    97    76    27   NA
library(GGally)
suppressWarnings(ggpairs(Y))
```



Gibbs sampling according to the 3-step scheme described above is implemented below. For priors, we set $\mu_0 = (120, 64, 26, 26)$ assuming we know these are the national averages of the health measurements. We then (waving our hands a little) select prior variances that keep these measurements mostly around zero and only lightly centered around our estimates.

```
# Gibbs sampling
n = nrow(Y)
p = ncol(Y)
mu0 = c(120, 64, 26, 26)
sd0 = mu0 / 2
# Setting prior on covariance
L0 = matrix(.1, p, p)
diag(L0) = 1
L0 = L0 * outer(sd0, sd0)
print(L0)
#>      [,1] [,2] [,3] [,4]
#> [1,] 3600 192.0 78.0 78.0
#> [2,] 192 1024.0 41.6 41.6
#> [3,] 78 41.6 169.0 16.9
#> [4,] 78 41.6 16.9 169.0

# Variance prior lightly concentrated, where nu0 > p is nu0 = p + 2
nu0 = p + 2
# Scale matrix - set to be the same as L0
S0 = L0

# Set starting values
Sigma = S0
Y.full = Y
O = 1 * (!is.na(Y)) # Get NOT NAs, coerce to int via * 1
for (j in 1:p) { # Looping through columns
  # For missing values in column, set to mean of the observed values
  mean.wo.na = mean(Y.full[, j], na.rm = TRUE)
  # Rows: all of those that are NA, and the jth column, set it to this mean
  Y.full[is.na(Y.full[, j]), j] = mean.wo.na
}
```

```

# Gibbs
THETA = SIGMA = Y.MISS = NULL
set.seed(1)
for (s in 1:1000) {
  # Update theta: step 1 of p.117 which is the same as previous
  ybar = colMeans(Y.full)
  Ln = inv(inv(L0) + n * inv(Sigma))
  mun = Ln %>% (inv(L0) %>% mu0 + n * inv(Sigma) %>% ybar)
  theta = mvrnorm(1, mun, Ln)

  # Update sigma: step 2 of p.117, same as previous
  resid = t(Y.full) - c(theta)
  Stheta = resid %>% t(resid)
  Sn = S0 + Stheta
  Sigma = inv(rWishart(1, nu0 + n, inv(Sn))[, , 1])

  # Update ymiss: step 3 of p.117, requires eqs 7.10, 7.11
  # Loop through rows of data (takes longer!!), need to sample rows individually
  # (independent) top of p.118
  for (i in 1:n) {
    # Skip if we already have it
    if (all(O[i, ] == 1)) {
      next
    }
    # Partition b = NA rows, a = present rows
    # Still works if a, b are empty, I presume
    oi = O[i, ]
    a = oi == 1
    b = oi == 0

    # Now we want to sample yb | ya, Sigma, Theta

    # \Sigma_{[a, a]}^{-1}, used in eqs 7.10 AND 7.11 (so calc once)
    iSa = inv(Sigma[a, a])
    # Calculate \Sigma_{[b, a]}(\Sigma_{[a, a]})^{-1} used in 7.10 AND 7.11
    beta.j = Sigma[b, a] %>% iSa
    # Calculate Sigma.j (7.11). Start with covariance matrix for the missing
    # vars, then influence by beta (decrease variance)
    Sigma.j = Sigma[b, b] - beta.j %>% Sigma[a, b]
    # Calculate theta.j (7.10). Start with standard theta, then change based on
    # residuals of other vals and what we know about covariances
    yi = Y.full[i, ]
    theta.j = theta[b] + beta.j %>% t(yi[a] - theta[a])

    # Now we have samples for subset of b. Preserve order, now sample
    Y.full[i, b] = mvrnorm(1, theta.j, Sigma.j)
  }

  # Concat
  THETA = rbind(THETA, theta)
  SIGMA = rbind(SIGMA, c(Sigma))
  Y.MISS = rbind(Y.MISS, Y.full[O == 0])
}

# Means and confidence intervals for posterior means
colnames(Y)
#> [1] "glu" "bp" "skin" "bmi"

```

```

colMeans(THETA)
#> [1] 123.51682 71.04978 29.38495 32.18491
apply(THETA, MARGIN = 2, FUN = function(d) quantile(d, prob = c(0.025, 0.5, 0.975)))
#>      [,1]      [,2]      [,3]      [,4]
#> 2.5% 118.7039 69.41573 27.62746 31.33561
#> 50%  123.5534 71.05000 29.39057 32.18047
#> 97.5% 127.8140 72.63692 31.05039 33.04161

# Sample correlation matrices
COR = array(dim = c(p, p, 1000))
for (s in 1:nrow(SIGMA)) {
  # It's in rows right now, refold into matrix
  Sig = matrix(SIGMA[s, ], nrow = p, ncol = p)
  # Calculate correlation matrix: (TODO: figure out how this works)
  COR[, , s] = Sig / sqrt(diag(Sig) %o% diag(Sig))
}

# Mean correlations from the correlation matrix samples. Can also calculate
# confidence intervals with quantile
apply(COR, MARGIN = c(1, 2), FUN = mean)
#>      [,1]      [,2]      [,3]      [,4]
#> [1,] 1.0000000 0.2236531 0.2475843 0.1888097
#> [2,] 0.2236531 1.0000000 0.2478914 0.2349531
#> [3,] 0.2475843 0.2478914 1.0000000 0.6510103
#> [4,] 0.1888097 0.2349531 0.6510103 1.0000000

```

7 Exercises

7.1 7.1

7.1.1 a

Since the density is uniform with respect to θ , the integral over the support of this function is infinite and cannot be 1.

7.1.2 b

$$p_J(\theta, \Sigma \mid y_1, \dots, y_n) \propto p(\theta, \Sigma) \times p(y_1, \dots, y_n \mid \theta, \Sigma) \quad (37)$$

$$\propto [|\Sigma|^{-(p+2)/2}] \times [|\Sigma|^{-n/2} \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1}))] \quad (38)$$

$$\propto |\Sigma|^{-(p+n+2)/2} \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1})/2) \quad (39)$$

To obtain the full conditionals of a parameter, we treat the other parameters as constant, so

$$p_J(\theta \mid y_1, \dots, y_n, \Sigma) \propto \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1})/2) \quad (40)$$

$$= \exp\left(-\sum_{i=1}^n (y_i - \theta)^T \Sigma^{-1} (y_i - \theta)/2\right) \quad \text{Expand back} \quad (41)$$

$$= \exp(-n(\bar{y} - \theta)^T \Sigma^{-1} (\bar{y} - \theta)/2) \quad (42)$$

$$= \text{dnormal}(\bar{y}, \Sigma/n) \quad (43)$$

and

$$p_J(\Sigma \mid y_1, \dots, y_n, \theta) \propto |\Sigma|^{-(p+n+2)/2} \exp(-\text{tr}(\mathbf{S}_\theta \Sigma^{-1})/2) \quad (44)$$

$$\propto \text{dinverse-wishart}(n+1, \mathbf{S}_\theta^{-1}) \quad (45)$$

7.2 7.3

```
bluecrab = as.matrix(read.table('Exercises/bluecrab.dat'))
orangecrab = as.matrix(read.table('Exercises/orangecrab.dat'))
```

7.2.1 a

```
crab.mcmc = lapply(list('bluecrab' = bluecrab, 'orangecrab' = orangecrab), function(crab) {
  p = ncol(crab)
  n = nrow(crab)
  ybar = colMeans(crab)

  # Prior parameters

  mu0 = ybar
  lambda0 = s0 = cov(crab)
  nu0 = 4

  S = 10000
  THETA = matrix(nrow = S, ncol = p)
  SIGMA = array(dim = c(p, p, S))

  # Start with sigma sample
  sigma = s0

  # Gibbs sampling
  library(MASS)

  # Also, inv = solve to make it more readable
  inv = solve

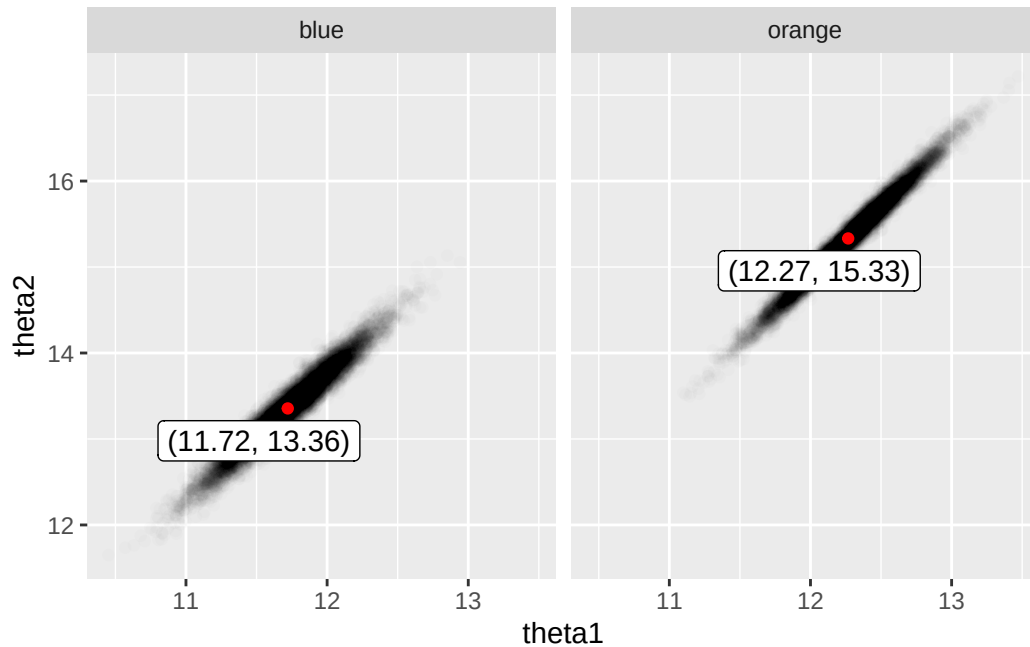
  for (s in 1:S) {
    # Update theta
    lambdan = inv(inv(lambda0) + n * inv(sigma))
    mun = lambdan %*% (inv(lambda0) %*% mu0 + n * inv(sigma) %*% ybar)
    theta = mvrnorm(n = 1, mun, lambdan)

    # Update sigma
    resid = t(crab) - c(theta)
    stheta = resid %*% t(resid)
    sn = s0 + stheta
    sigma = inv(rWishart(1, nu0 + n, inv(sn))[, , 1])

    THETA[s, ] = theta
    SIGMA[, , s] = sigma
  }

  list(theta = THETA, sigma = SIGMA)
})
```

7.2.2 b

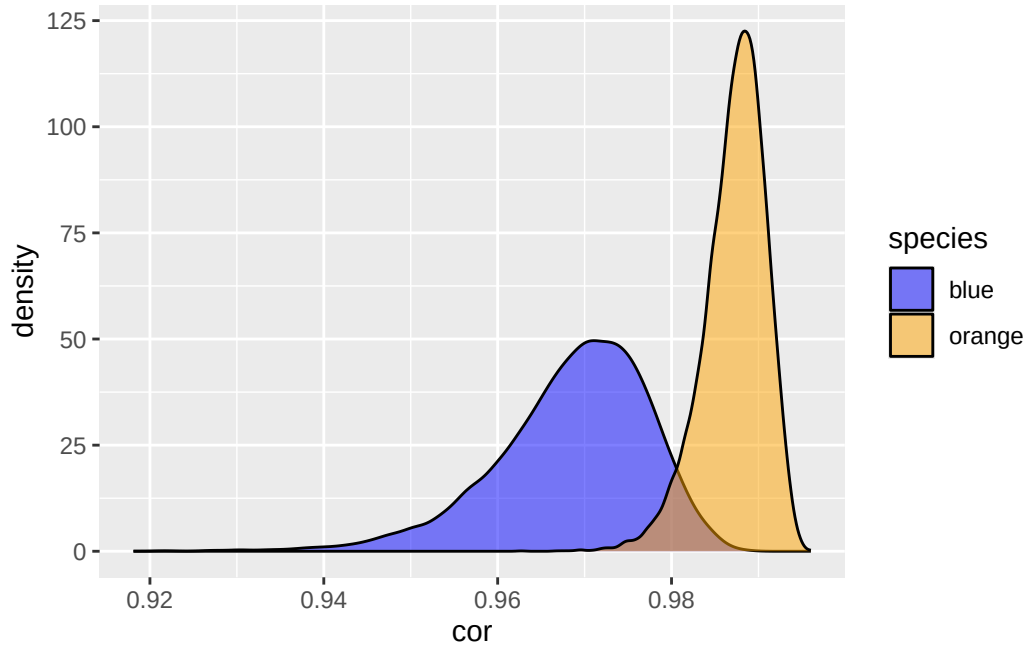


There is strong evidence that orange crabs tend to be larger in both measurements than blue crabs:

```
mean(orangecrab.df$theta1 > bluecrab.df$theta1)
#> [1] 0.9013
mean(orangecrab.df$theta2 > bluecrab.df$theta2)
#> [1] 0.9979
```

7.2.3 c

```
bluecrab.cor = apply(crab.mcmc$bluecrab$sigma, MARGIN = 3, FUN = function(covmat) {
  covmat[1, 2] / (sqrt(covmat[1, 1] * covmat[2, 2]))
})
orangecrab.cor = apply(crab.mcmc$orangecrab$sigma, MARGIN = 3, FUN = function(covmat) {
  covmat[1, 2] / (sqrt(covmat[1, 1] * covmat[2, 2]))
})
cor.df = data.frame(species = c(rep('blue', length(bluecrab.cor)), rep('orange', length(orangecrab.cor))),
  cor = c(bluecrab.cor, orangecrab.cor))
ggplot(cor.df, aes(x = cor, fill = species)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c('blue', 'orange'))
```

```
mean(bluecrab.cor < orangecrab.cor)
#> [1] 0.9899
```

The orange crab species appears to have a much higher correlation between its two measurements than the blue crab species.

7.3 7.4

```
agehw = as.matrix(read.table('Exercises/agehw.dat'))
colnames(agehw) = agehw[1, ]
agehw = agehw[-1, ]
agehw = matrix(as.numeric(agehw), nrow = 100)
```

7.3.1 a

With typical intuitions about life expectancy and age of marriage, I suspect that the ages of most of the married couples will fall between 25 and 80. There may be slight age differences among men and women, but nothing that I'm confident enough to encode in my prior. Thus I will set $\mu_0 = ((25 + 80)/2, (25 + 80)/2) = (52.5, 52.5)^T$

I have no choice but to pick a semiconjugate prior distribution in this case, but it does seem intuitive that there are less married couples at ages 25 and 80 than there are married couples around age 50, thus justifying a bell curve centered around 52.5 with variance $13.75^2 \approx 189$ such that approximately 95% of my prior falls within the range (25, 80).

I also have reason to believe the ages of the couples are quite tightly correlated, so knowing the above variance, I will aim for a prior correlation of 0.75. Solving the correlation equation gives

$$0.75 = \frac{\sigma_{1,2}}{189} \quad (46)$$

$$\Rightarrow \sigma_{1,2} = 141.75 \quad (47)$$

So I will set

$$\Lambda_0 = \begin{bmatrix} 189 & 141.75 \\ 141.75 & 189 \end{bmatrix}$$

Like previous problems, for the variance, I will set $\mathbf{S}_0^{-1} = \Lambda_0$ and $\nu_0 = p + 2 = 4$. Note that this only loosely centers our covariance matrix prior on Λ_0 , so I am being a bit conservative in terms of my belief in the variance and the correlation between the ages.

```
Y = agehw
p = ncol(agehw)
n = nrow(agehw)
ybar = colMeans(agehw)

mu0 = rep(52.5, p)
lambda0 = s0 = rbind(c(189, 141.75), c(141.75, 189))
# nu0 = p + 2
nu0 = p + 2 + 10
```

7.3.2 b

The wording of the question is interesting - I assume I'm supposed to sample a fixed θ, Σ and from there sample 100 points all with the same parameters. If I were to do this myself, I feel like I would sample a new data point for each sample of θ, Σ ...

In fact, because of that wording, I originally set $\nu_0 = p + 2 = 4$ to loosely center my prior. But given that this variance will often produce uncorrelated prior predictive datasets, I'm increasing ν_0 a bit...

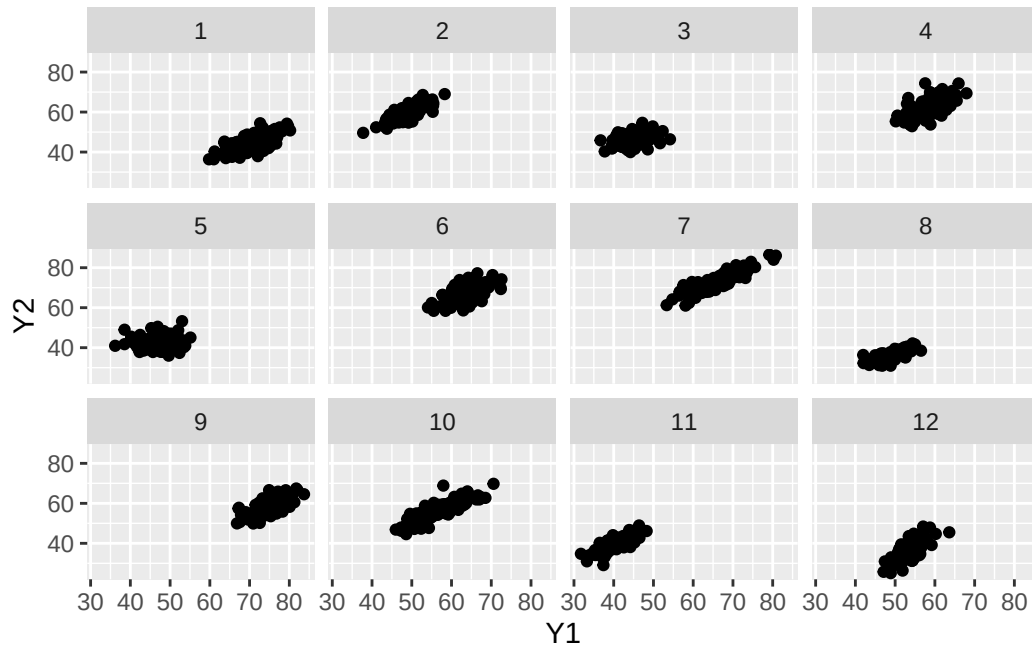
After increasing ν_0 , I'm fairly comfortable with what these posterior predictive datasets look like.

```
N = 100
S = 12

Y_preds = lapply(1:S, function(s) {
  # Sample THETA according to prior
  theta = mvrnorm(n = 1, mu0, lambda0)
  sigma = inv(rWishart(1, nu0, inv(s0))[, , 1])
  Y_s = mvrnorm(n = 100, theta, sigma)
  data.frame(Y1 = Y_s[, 1], Y2 = Y_s[, 2], dataset = s)
})

Y_comb = do.call(rbind, Y_preds)

ggplot(Y_comb, aes(x = Y1, y = Y2)) +
  geom_point() +
  facet_wrap(~ dataset)
```



7.3.3 c

```
S = 10000

# Reuse this since we'll need to specify different priors
do_mcmc = function(Y, mu0, lambda0, s0, nu0) {
  ybar = colMeans(Y)
  p = ncol(Y)
  n = nrow(Y)

  THETA = matrix(nrow = S, ncol = p)
  SIGMA = array(dim = c(p, p, S))

  # Start with sigma sample
  sigma = cov(Y)

  # Gibbs sampling

  # Also, inv = solve to make it more readable
  inv = solve

  for (s in 1:S) {
    # Update theta
    lambdan = inv(inv(lambda0) + n * inv(sigma))
    mun = lambdan %*% (inv(lambda0) %*% mu0 + n * inv(sigma) %*% ybar)
    theta = mvrnorm(n = 1, mun, lambdan)

    # Update sigma
    resid = t(Y) - c(theta)
    stheta = resid %*% t(resid)
    sn = s0 + stheta
    sigma = inv(rWishart(1, nu0 + n, inv(sn))[, , 1])

    THETA[s, ] = theta
    SIGMA[, , s] = sigma
  }
}
```

```

}

list(theta = THETA, sigma = SIGMA)
}

my_prior_mcmc = do_mcmc(agehw, mu0, lambda0, s0, nu0)
THETA = my_prior_mcmc$theta
SIGMA = my_prior_mcmc$sigma

# For reuse later
print_quantiles = function(THETA, SIGMA) {
  print("Husband")
  print(quantile(THETA[, 1], probs = c(0.025, 0.5, 0.975))) # Husband
  print("Wife")
  print(quantile(THETA[, 2], probs = c(0.025, 0.5, 0.975))) # Wife
  cors = apply(SIGMA, MARGIN = 3, FUN = function(covmat) {
    covmat[1, 2] / (sqrt(covmat[1, 1] * covmat[2, 2]))
  })
  print("Correlation")
  print(quantile(cors, probs = c(0.025, 0.5, 0.975)))
}

print_quantiles(THETA, SIGMA)
#> [1] "Husband"
#>      2.5%      50%      97.5%
#> 42.01685 44.52494 47.03880
#> [1] "Wife"
#>      2.5%      50%      97.5%
#> 38.60299 40.99994 43.42255
#> [1] "Correlation"
#>      2.5%      50%      97.5%
#> 0.8615416 0.9028502 0.9321153

```

7.3.4 d

I haven't done 7.2, but doing Jeffreys' prior and a "diffuse prior" below will still be helpful to see what effect prior information has.

i

```

THETA = matrix(nrow = S, ncol = p)
SIGMA = array(dim = c(p, p, S))

# Start with sigma sample
sigma = cov(Y)

# Gibbs sampling

# Also, inv = solve to make it more readable
inv = solve

for (s in 1:S) {
  # Update theta
  theta = mvrnorm(n = 1, ybar, sigma / n)

  # Update sigma
  resid = t(Y) - c(theta)
  stheta = resid %*% t(resid)
}

```

```

sigma = inv(rWishart(1, n + 1, inv(stheta))[, , 1])

THETA[s, ] = theta
SIGMA[, , s] = sigma
}

print_quantiles(THETA, SIGMA)
#> [1] "Husband"
#>      2.5%      50%      97.5%
#> 41.73803 44.42889 47.15085
#> [1] "Wife"
#>      2.5%      50%      97.5%
#> 38.37376 40.91439 43.48397
#> [1] "Correlation"
#>      2.5%      50%      97.5%
#> 0.8608382 0.9038100 0.9347940

```

ii

Unit information prior (skipping)

iii

```

mu0 = rep(0, p)
lambda0 = 10^5 * diag(p)
s0 = 1000 * diag(p)
nu0 = 3
diffuse_mcmc = do_mcmc(agehw, mu0, lambda0, s0, nu0)
print_quantiles(diffuse_mcmc$theta, diffuse_mcmc$sigma)
#> [1] "Husband"
#>      2.5%      50%      97.5%
#> 41.67960 44.41405 47.18645
#> [1] "Wife"
#>      2.5%      50%      97.5%
#> 38.32275 40.88109 43.51451
#> [1] "Correlation"
#>      2.5%      50%      97.5%
#> 0.7931776 0.8552845 0.8996432

```

7.3.5 e

It doesn't really seem like the prior information matters because the sample size is so large. Regardless of whether the prior is informative, the quantiles and correlations end up quite similar. Maybe the diffuse prior is slightly different, but it's not a big difference.

If we have a smaller sample size, this may be different. Let's see what happens if I truncate the dataset to the first 25 variables and rerun with my informative prior and the diffuse prior:

My prior

```

mu0 = rep(52.5, p)
lambda0 = s0 = rbind(c(189, 141.75), c(141.75, 189))
# nu0 = p + 2
nu0 = p + 2 + 10
my_prior_mcmc_short = do_mcmc(agehw[1:25, ], mu0, lambda0, s0, nu0)
print_quantiles(my_prior_mcmc_short$theta, my_prior_mcmc_short$sigma)
#> [1] "Husband"
#>      2.5%      50%      97.5%
#> 40.96477 45.37803 49.83724

```

```
#> [1] "Wife"
#>      2.5%      50%      97.5%
#> 38.41850 43.04898 47.77481
#> [1] "Correlation"
#>      2.5%      50%      97.5%
#> 0.8418117 0.9136244 0.9540250
```

Diffuse prior

```
mu0 = rep(0, p)
lambda0 = 10^5 * diag(p)
s0 = 1000 * diag(p)
nu0 = 3
diffuse_mcmc_short = do_mcmc(agehw[1:25, ], mu0, lambda0, s0, nu0)
print_quantiles(diffuse_mcmc_short$theta, diffuse_mcmc_short$sigma)
#> [1] "Husband"
#>      2.5%      50%      97.5%
#> 39.07635 45.12302 51.00340
#> [1] "Wife"
#>      2.5%      50%      97.5%
#> 36.53189 42.74457 48.81278
#> [1] "Correlation"
#>      2.5%      50%      97.5%
#> 0.5427696 0.7613713 0.8823101
```

In this case, the effect of the prior on correlation especially is easily observed. The correlation for the diffuse prior is quite low, as it is being dragged towards nothing.